

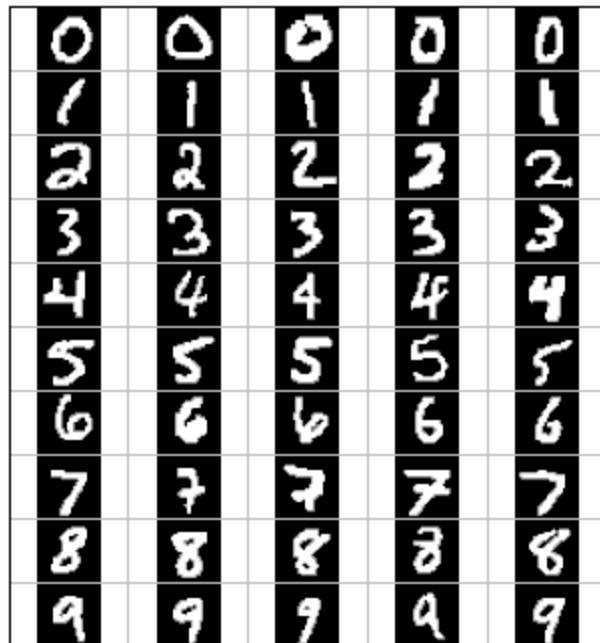
CSCI 4150 - Intro to AI - Spring 2025



Acknowledgements

This project is part of the [Pac-man projects](#) created by [John DeNero](#) and [Dan Klein](#).

Optional Machine Learning Project - Naive Bayes



Which Digit?



Which are Faces?

Introduction

In this project, you will design a Naive Bayes classifier. You will test your classifiers on two image data sets: a set of scanned handwritten digit images and a set of face images in which edges have already been detected. Even with simple features, your classifiers will be able to do quite well on these tasks when given enough training data.

Optical character recognition (**OCR**) is the task of extracting text from image sources. The first data set on which you will run your classifiers is a collection of handwritten numerical digits (0-9). This is a very commercially useful technology, similar to the technique used by the US post office to route mail by zip codes. There are systems that can perform with over 99% classification accuracy.

Face detection is the task of localizing faces within video or still images. The faces can be at any location and vary in size. There are many applications for face detection, including human computer interaction and surveillance. You will attempt a simplified face detection task in which your system is presented with an image that has been pre-processed by an edge detection algorithm. The task is to determine whether the edge image is a face or not. There are several systems in use that perform quite well at the face detection task.

The code for this project includes the following files and data, available as a [zip file](#) (please note that this has code for other classifiers that you may ignore).

Data file

[data.zip](#) Data file, including the digit and face data.

Files you will edit

[naiveBayes.py](#) The location where you will write your naive Bayes classifier.

[dataClassifier.py](#) The wrapper code that will call your classifiers. You will use this code to analyze the behavior of your classifier and may edit some parts of it if you want to.

[answers.py](#) Answers to Question 2 and Question 4 go here.

Files you should read but NOT edit

[classificationMethod.py](#) Abstract super class for the classifiers you will write.
(You **should** read this file carefully to see how the infrastructure is set up.)

[samples.py](#) I/O code to read in the classification data.

`util.py`

Code defining some useful tools. You may be familiar with some of these by now, and they will save you a lot of time.

`mostFrequent.py`

A simple baseline classifier that just labels every instance as the most frequent class.

`runMinicontest.py`

The command you will use to run the minicontest, if you decide to enter.

What to submit: You will fill in portions of `answers.py`, `naiveBayes.py`, and `dataClassifier.py` (only) during the assignment, and submit them.

Evaluation: Your code will be autograded for technical correctness. Please *do not* change the names of any provided functions or classes within the code, or you will wreak havoc on the autograder.

Getting Started

To try out the classification pipeline, run `dataClassifier.py` from the command line. This will classify the digit data using the default classifier (`mostFrequent`) which blindly classifies every example with the most frequent label.

```
python dataClassifier.py
```

As usual, you can learn more about the possible command line options by running:

```
python dataClassifier.py -h
```

We have defined some simple features for you. Later you will design some better features. Our simple feature set includes one feature for each pixel location, which can take values 0 or 1 (off or on). The features are encoded as a `Counter` where keys are feature locations (represented as (column,row)) and values are 0 or 1. The face recognition data set has value 1 only for those pixels identified by a Canny edge detector.

Implementation Note: You'll find it easiest to hard-code the binary feature assumption. If you do, make sure you don't include any non-binary features. Or, you can write your code more generally, to handle arbitrary feature values, though this will probably involve a preliminary pass through the training set to find all possible feature values (and

you'll need an "unknown" option in case you encounter a value in the test data you never saw during training).

Naive Bayes

A skeleton implementation of a naive Bayes classifier is provided for you in `naiveBayes.py`. You will fill in the `trainAndTune` function, the `calculateLogJointProbabilities` function and the `findHighOddsFeatures` function.

Theory

A naive Bayes classifier models a joint distribution over a label Y and a set of observed random variables, or *features*, $(F_1, F_2, \dots, F_n)$, using the assumption that the full joint distribution can be factored as follows (features are conditionally independent given the label):

$$P(F_1, \dots, F_n, Y) = P(Y) \prod_i P(F_i | Y)$$

To classify a datum, we can find the most probable label given the feature values for each pixel, using Bayes theorem:

$$\begin{aligned} P(y | f_1, \dots, f_m) &= \frac{P(f_1, \dots, f_m | y) P(y)}{P(f_1, \dots, f_m)} \\ &= \frac{P(y) \prod_{i=1}^m P(f_i | y)}{P(f_1, \dots, f_m)} \\ \arg \max_y P(y | f_1, \dots, f_m) &= \arg \max_y \frac{P(y) \prod_{i=1}^m P(f_i | y)}{P(f_1, \dots, f_m)} \\ &= \arg \max_y P(y) \prod_{i=1}^m P(f_i | y) \end{aligned}$$

Because multiplying many probabilities together often results in underflow, we will instead compute **log probabilities** which have the same argmax:

$$\begin{aligned} \arg \max_y \log P(y | f_1, \dots, f_m) &= \arg \max_y \log P(y, f_1, \dots, f_m) \\ &= \arg \max_y \left\{ \log P(y) + \sum_{i=1}^m \log P(f_i | y) \right\} \end{aligned}$$

To compute logarithms, use `math.log()`, a built-in Python function.

Parameter Estimation

Our naive Bayes model has several parameters to estimate. One parameter is the **prior distribution** over labels (digits, or face/not-face), $P(Y)$.

We can estimate $P(Y)$ directly from the training data:

$$\hat{P}(y) = \frac{c(y)}{n}$$

where $c(y)$ is the number of training instances with label y and n is the total number of training instances.

The other parameters to estimate are the **conditional probabilities** of our features given each label y : $P(F_i|Y = y)$. We do this for each possible feature value ($f_i \in \{0, 1\}$).

$$\hat{P}(F_i = f_i|Y = y) = \frac{c(f_i, y)}{\sum_{f'_i \in \{0,1\}} c(f'_i, y)}$$

where $c(f_i, y)$ is the number of times pixel F_i took value f_i in the training examples of label y .

Smoothing

Your current parameter estimates are *unsmoothed*, that is, you are using the empirical estimates for the parameters $P(f_i|y)$. These estimates are rarely adequate in real systems. Minimally, we need to make sure that no parameter ever receives an estimate of zero, but good smoothing can boost accuracy quite a bit by reducing overfitting.

In this project, we use *Laplace smoothing*, which adds k counts to every possible observation value:

$$P(F_i = f_i|Y = y) = \frac{c(f_i, y) + k}{\sum_{f'_i \in \{0,1\}} (c(f'_i, y) + k)}$$

If $k=0$, the probabilities are unsmoothed. As k grows larger, the probabilities are smoothed more and more. You can use your validation set to determine a good value for k . **Note:** don't smooth $P(Y)$.

Question 1 (6 points) Implement `trainAndTune` and `calculateLogJointProbabilities` in `naiveBayes.py`. In `trainAndTune`, estimate conditional probabilities from the training data for each possible value of k given in the list `kgrid`. Evaluate accuracy on the held-out validation set for each k and choose the value with the highest validation accuracy. In case of ties, prefer the *lowest* value of k . Test your classifier with:

```
python dataClassifier.py -c naiveBayes --autotune
```

Hints and observations:

- The method `calculateLogJointProbabilities` uses the conditional probability tables constructed by `trainAndTune` to compute the log posterior probability for each label y given a feature vector. The comments of the method describe the data structures of the input and output.
- You can add code to the `analysis` method in `dataClassifier.py` to explore the mistakes that your classifier is making. This is optional.
- When trying different values of the smoothing parameter k , think about the number of times you scan the training data. Your code should save computation by avoiding redundant reading.
- To run your classifier with only one particular value of k , remove the `--autotune` option. This will ensure that `kgrid` has only one value, which you can change with `-k`.
- Using a fixed value of $k=2$ and 100 training examples, you should get a validation accuracy of about 69% and a test accuracy of 55%.
- Using `--autotune`, which tries different values of k , you should get a validation accuracy of about 74% and a test accuracy of 65%.
- Accuracies may vary slightly because of implementation details. For instance, ties are not deterministically broken in the `Counter.argmax()` method.
- To run on the face recognition dataset, use `-d faces` (optional).

Odds Ratios

One important tool in using classifiers in real domains is being able to inspect what they have learned. One way to inspect a naive Bayes model is to look at the most likely features for a given label.

Another, better, tool for understanding the parameters is to look at *odds ratios*. For each pixel feature F_i and classes y_1, y_2 , consider the odds ratio:

$$\text{odds}(F_i = 1, y_1, y_2) = \frac{P(F_i = 1|y_1)}{P(F_i = 1|y_2)}$$

This ratio will be greater than one for features which cause belief in y_1 to increase relative to y_2 .

The features that have the greatest impact at classification time are those with both a high probability (because they appear often in the data) and a high odds ratio (because they strongly bias one label versus another).

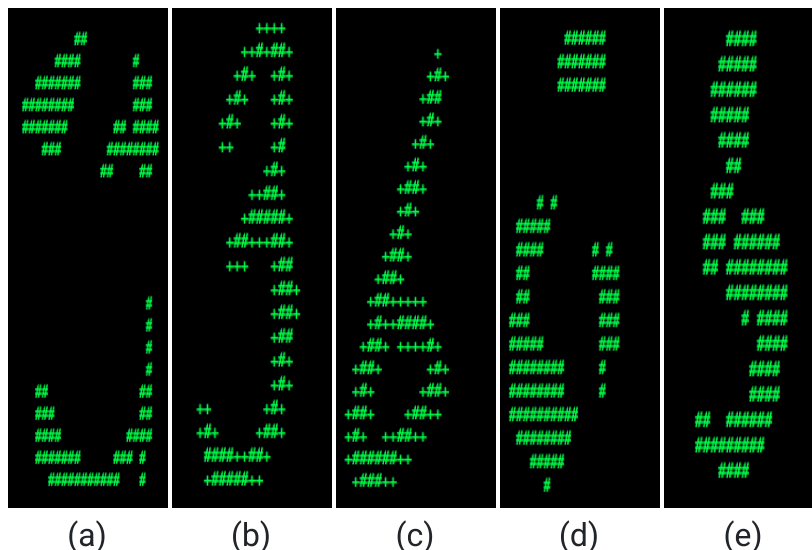
To run the autograder for this question:

```
python autograder.py -q q1
```

Question 2 (2 points) Fill in the function `findHighOddsFeatures(self, label1, label2)`. It should return a list of the 100 features with highest odds ratios for `label1` over `label2`. The option `-o` activates an odds ratio analysis. Use the options `-1 label1 -2 label2` to specify which labels to compare. Running the following command will show you the 100 pixels that best distinguish between a 3 and a 6.

```
python dataClassifier.py -a -d digits -c naiveBayes -o -1 3 -2 6
```

Use what you learn from running this command to answer the following question. Which of the following images best shows those pixels which have a high odds ratio with respect to 3 over 6? (That is, which of these is most like the output from the command you just ran?)



To answer: please return 'a', 'b', 'c', 'd', or 'e' from the function `q2` in `answers.py`.

Hints:

- To avoid divide-zero errors you may want to use smoothing when computing the odds ratio.